

INFORME SOCKET CON HILOS – MALANCA BRUNELLA

Este proyecto representa una evolución significativa respecto al trabajo práctico anterior (Socket Monohilo). Mientras que en la versión anterior el servidor quedaba bloqueado atendiendo a un único cliente, esta nueva implementación utiliza la clase Thread de Java para gestionar conexiones simultáneas. La arquitectura ahora permite que múltiples usuarios interactúen en tiempo real sin interferir en los procesos de los demás, transformando un canal de eco simple en un sistema de mensajería inteligente.

Para lograr esta funcionalidad, el servidor se dividió en una estructura principal y una clase interna dedicada a cada cliente:

A. El Servidor y la Gestión de Hilos

En la clase Servidor, el cambio fundamental respecto al modelo monohilo reside en el bucle while(true) dentro del main.

```
11
12 public static void main(String[] args) {
13     System.out.println("[LOG] === SERVIDOR MULTITHILO INICIADO EN PUERTO " + PUERTO + " ===");
14
15     try (ServerSocket serverSocket = new ServerSocket(PUERTO)) {
16         while (true) {
17             Socket socket = serverSocket.accept();
18             new Thread(new ManejadorCliente(socket)).start();
19         }
20     } catch (IOException e) {
21         System.err.println("[LOG] Error crítico: " + e.getMessage());
22     }
23 }
```

En esta captura se observa cómo el ServerSocket acepta la conexión y, en lugar de procesarla, crea un new Thread(new ManejadorCliente(socket)).start();. Esto es lo que permite que el servidor no se bloquee.

B. Análisis de Métodos y Estructura del Código

Para que el servidor sepa a quién enviar cada mensaje, utilicé un Map que asocia nombres de usuario con sus respectivos hilos de ejecución. Para administrar múltiples clientes de forma segura, se utilizó un ConcurrentHashMap. Esta estructura es vital en entornos multihilo para evitar que la lista de usuarios se corrompa cuando varios hilos intentan leer o escribir al mismo tiempo.

```
public class Servidor {
    private static final int PUERTO = 5000;
    private static final Map<String, ManejadorCliente> clientesConectados = new ConcurrentHashMap<>();
}
```

En esta sección se observa la declaración del mapa clientesConectados, que asocia el nombre de cada usuario con su hilo correspondiente (ManejadorCliente).

Para que el ruteo de mensajes privados funcione, cada usuario debe tener un identificador exclusivo. Se implementó una lógica de validación que evita nombres duplicados.

```
97 private String validarNombre(String nombreBase) {
98     String resultado = nombreBase;
99     int i = 1;
100     while (clientesConectados.containsKey(resultado)) {
101         resultado = nombreBase + i;
102         i++;
103     }
104     if (!resultado.equals(nombreBase)) {
105         salida.println("[AVISO] El nombre ya existía. Se te asignó: " + resultado);
106     }
107     return resultado;
108 }
```

El método `validarNombre` verifica si el nombre elegido ya está en uso; de ser así, añade un sufijo numérico automáticamente, garantizando la integridad de la lista de conexiones.

C. Lógica de Mensajería (ProcesarMensajeEspecial)

Este método es el núcleo del TP. Reemplaza la simple respuesta de "eco" por un sistema de enrutamiento.

```
122 private void procesarMensajeEspecial(String input) {
123     try {
124         int primerEspacio = input.indexOf(" ");
125         if (primerEspacio == -1) {
126             responder("[ERROR] Formato incorrecto. Usa *User Mensaje");
127             return;
128         }
129
130         String cabecera = input.substring(1, primerEspacio).trim();
131         String contenido = input.substring(primerEspacio).trim();
132
133         if (cabecera.equalsIgnoreCase("ALL")) {
134             clientesConectados.forEach((nombre, manejador) -> {
135                 if (!nombre.equals(this.nombreUsuario)) {
136                     manejador.salida.println("[TODOS] " + this.nombreUsuario + ": " + contenido);
137                 }
138             });
139         } else {
140             String[] destinatarios = cabecera.split(",");
141             for (String d : destinatarios) {
142                 String dLimpio = d.trim();
143                 if (dLimpio.startsWith("***")) dLimpio = dLimpio.substring(1);
144
145                 // BÚSQUEDA INSENSIBLE A MAYÚSCULAS
146                 ManejadorCliente receptor = null;
147                 for (String nombreConectado : clientesConectados.keySet()) {
148                     if (nombreConectado.equalsIgnoreCase(dLimpio)) {
149                         receptor = clientesConectados.get(nombreConectado);
150                         break;
151                     }
152                 }
153
154                 if (receptor != null) {
155                     receptor.salida.println("[PRIVADO de " + this.nombreUsuario + "]: " + contenido);
156                     System.out.println("[LOG-PRIVADO] " + this.nombreUsuario + " para " + dLimpio + ": " + contenido);
157                 } else {
158                     // No se encontró destinatario
159                 }
160             }
161         }
162     } catch (Exception e) {
163         // Manejo de excepciones
164     }
165 }
```

Aquí se ve la lógica de los mensajes privados (*user) y globales (*ALL). El servidor analiza la cadena recibida, busca al destinatario en el Mapa y redirige el flujo de datos únicamente hacia los sockets correspondientes.

D. Funcionalidades de Procesamiento (Consonantes y Hora)

El servidor demuestra su capacidad de procesamiento independiente para cada cliente.

```
167 private String obtenerConsonantes(String palabra) {
168     String vocales = "aeiouáéíóúüAEIOUÁÉÍÓÚÜ";
169     StringBuilder todas = new StringBuilder();
170     LinkedHashSet<Character> unicas = new LinkedHashSet<>();
171     int cant = 0;
172     for (char c : palabra.toCharArray()) {
173         if (Character.isLetter(c) && vocales.indexOf(c) == -1) {
174             todas.append(c).append(" ");
175             unicas.add(Character.toLowerCase(c));
176             cant++;
177         }
178     }
179     return "Palabra: " + palabra + " | Consonantes: " + todas + " | Únicas: " + unicas + " | Total: " + cant;
180 }

else if (msgUpper.equals("HORA")) {
    DateTimeFormatter dtf = DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm:ss");
    responder("Fecha/Hora actual: " + dtf.format(LocalDate.now()));
}
```

Se observa el algoritmo de filtrado de caracteres y el uso de LocalDateTime. Es importante notar que cada cliente puede solicitar estos datos simultáneamente sin interferir en la respuesta de los demás.

MONOHILO VS MULTHILO

Característica	Práctica Sin Hilos	Práctica Con Hilos (Actual)
Arquitectura	Secuencial (Bloqueante)	Concurrente (No bloqueante)
Capacidad	1 solo cliente	Clientes ilimitados (teóricos)
Rol del Servidor	Receptor pasivo	Administrador y Router de mensajes
Estructura de Datos	No necesaria	Uso de Mapas para gestión de sesiones

PRUEBAS DE FUNCIONAMIENTO

Para validar el sistema, se realizaron pruebas de estrés conectando tres clientes simultáneos.

```
:Output
PracticaSocketconHilos (run) x | PracticaSocketconHilos (run) #2 x | PracticaSocketconHilos (run) #3 x |
run:
[LOG] === SERVIDOR MULTIHILO INICIADO EN PUERTO 5000 ===
[LOG] [+] bru se ha unido.
[LOG] [+] mar se ha unido.
[LOG] [mar]: *ALL holaa
[LOG] [bru]: LISTA
[SERVIDOR para bru]: Usuarios conectados: [bru, mar]
[LOG] [bru]: HOLA
[SERVIDOR para bru]: [ERROR] Comando no reconocido o formato incorrecto.
[LOG] [bru]: LISTA
[SERVIDOR para bru]: Usuarios conectados: [bru, mar]
[LOG] [mar]: CONSONANTES computadora
[SERVIDOR para mar]: Palabra: computadora | Consonantes: c m p t d r | Únicas: [c, m, p, t, d, r] | Total: 6
[LOG] [mar]: SALIR
[SERVIDOR para mar]: Desconectando...
[LOG] [-] mar se ha ido.
```

Muestra cómo el servidor registra las entradas de diferentes usuarios (Ej: MAR, BRU) y

cómo monitoriza el intercambio de mensajes privados entre ellos, cumpliendo con la función de log de auditoría requerida.

CONCLUSIÓN

La transición de un modelo de socket simple a uno multihilo es esencial para aplicaciones de red modernas. La implementación de la interfaz Runnable y el manejo de colecciones concurrentes aseguran que el sistema sea estable y que la información llegue a su destino correcto en milisegundos, permitiendo una experiencia de usuario fluida y profesional.